PS5 · PLDs and decoders (Ch 6) — Worked Solutions

Source: Digital Logic homework for Oct. 23 · study-slides (vetame21.github.io/study-slides)

> [ADDED] The original problem sets have no solutions. These worked solutions were written for this site; every conversion and every Boolean result was checked in code (truth tables and an exact K-map minimizer), and the VHDL was compiled. Compare them with your lecturer's model answers when those are released.

1 Simplify and implement with a PLD; draw the circuit in compact representation. $F_1 = A'BCD + AC'D' + B'CD$, $F_2 = A'B + AB' + C'D + CD'$, $F_3 = (A + B + CD)(A' + B + C' + D) + A'B'C'$.

(a) Simplify F_1.

> **Answer:** $F_1 = A'CD + B'CD + AC'D'$

Minterms: $A'BCD = 7$, $AC'D' = 8, 12$, $B'CD = 3, 11$, so $F_1 = \Sigma(3, 7, 8, 11, 12)$. On the K-map, 3 and 7 pair → $A'CD$; 3 and 11 pair → $B'CD$; 8 and 12 pair → $AC'D'$. (Algebraically: $A'BCD + B'CD = CD(A'B + B') = CD(A' + B')$, which expands to $A'CD + B'CD$.)

Why this method: K-map on the minterm list; 15 is not in F_1, so CD alone can't be used.

(b) Simplify F_2.

> **Answer:** $F_2 = A'B + AB' + C'D + CD'$ $(= A \oplus B + C \oplus D)$

It is already minimal: each pair is an XOR, and the K-map (12 ones, zeros only at 0, 3, 12, 15) has no group of 4 or 8, so it needs four groups of 2.

Why this method: Check minimality with the K-map before 'simplifying' further.

(c) Simplify F_3.

> **Answer:** $F_3 = B + C' + D$

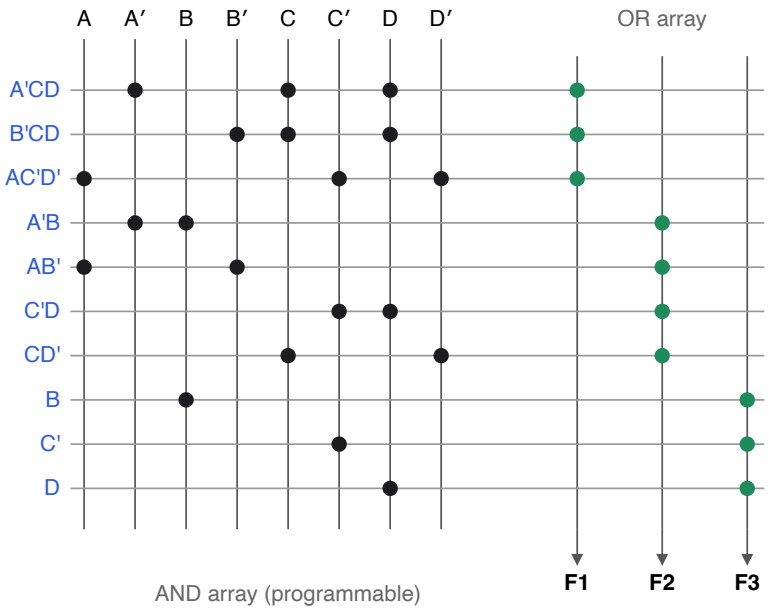
Use $(X + B)(Y + B) = B + XY$ with $X = A + CD$, $Y = A' + C' + D$: $XY = AC' + AD + A'CD + CD = AC' + AD + CD$. So $F_3 = B + AC' + AD + CD + A'B'C'$. Since $B + B'(A'C') = B + A'C'$: $F_3 = B + AC' + A'C' + AD + CD = B + C' + AD + CD$. Then $C' + CD = C' + D$, and D covers AD: $F_3 = B + C' + D$. Check: $F_3 = 0$ only for $B = 0, C = 1, D = 0$ (minterms 2 and 10).

Why this method: Factor the common B out of the product of sums first, then use $X + X'Y = X + Y$ twice.

(d) Draw the PLA in compact representation.

> **Answer:** 4 inputs, 10 product terms, 3 outputs (figure).

Each row of the AND array is one product term; a dot means 'this literal is connected'. Each column of the OR array is one output; a dot means 'this term is ORed in'. F_1 uses $A'CD, B'CD, AC'D'$; F_2 uses $A'B, AB', C'D, CD'$; F_3 uses the one-literal terms B, C', D. No term is shared here, so a PAL would work just as well.



Why this method: Compact (dot) notation from Ch 6: draw only the connections you keep.

---

2 Implement each function using a 4-to-16 decoder (active-low outputs, like the 74x154) and NAND gates.

(1) $F(A, B, C, D) = \Sigma(0, 4, 5, 8, 12, 13, 14)$

> **Answer:** One 7-input NAND of $\overline{Y_0}, \overline{Y_4}, \overline{Y_5}, \overline{Y_8}, \overline{Y_{12}}, \overline{Y_{13}}, \overline{Y_{14}}$

Inputs A, B, C, D drive the decoder's select inputs (A = MSB). Each active-low output $\overline{Y_i}$ is 0 exactly for minterm i. $F = Y_0 + Y_4 + \cdots + Y_{14} = (\overline{Y_0} \cdot \overline{Y_4} \cdots \overline{Y_{14}})'$ by DeMorgan, which is a NAND of those outputs.

Why this method: Decoder = minterm generator; OR of active-low lines = NAND.

(2) $F(A, B, C, D) = A'BCD + AC'D' + B'CD$

> **Answer:** $F = \Sigma(3, 7, 8, 11, 12)$: a 5-input NAND of $\overline{Y_3}, \overline{Y_7}, \overline{Y_8}, \overline{Y_{11}}, \overline{Y_{12}}$

First expand to a canonical sum: $A'BCD = 7$; $AC'D' \to 8, 12$; $B'CD \to 3, 11$. Then NAND the matching decoder outputs, exactly as in (1).

Why this method: Always convert to minterms first; the decoder can't use a simplified form.